

FAIDR-Fit: Exploring molecular features and function of disordered regions across the human proteome, in your browser

Guillaume Koenig*

2026-08-25

Introduction

Intrinsically disordered regions (IDRs) do not have a single stable conformation under physiological conditions. Instead, they exhibit an ensemble of conformations. The dynamic flexibility is important for protein function, including protein-protein interactions and the formation of biomolecular condensates. Significantly, IDRs are implicated in neuro-degenerative diseases. For example, TDP-43 has a low-complexity intrinsically disordered C-terminal region that naturally promotes self-association, but tips into pathological aggregation in ALS (Conicella et al. 2016). Thus, there are strong incentives to understand IDRs better, both in how they interact and in what their associated function might be.

The amino acid sequence of disordered regions is not conserved as much as for structured domains; the literature relies instead on molecular features over an entire IDR (Pritišanac et al. 2026; Ruff et al. 2026). Examples of molecular features are net charge, hydrophobicity, presence of particular motifs. Zarin et al. (2019) shows that molecular features of IDRs are conserved, by comparing against simulated IDRs evolved without selection pressure (using orthologs as basis).

One technique for finding IDR associated function is to first cluster IDRs based on similarity of their molecular features, and then look at what is common in proteins for them (Zarin et al. 2019; Ruff et al. 2026). Zarin et al. (2019) finds 23 clusters in the yeast proteome, and

*guillaume.koenig@mail.utoronto.ca

for each cluster checks which *Gene Ontology* terms (GO) their proteins are over-represented with, directly allowing to infer what function(s) the IDRs are associated with.

Another technique is bringing function first, and looking at molecular features after. King et al. (2024) starts from a set of nucleolus proteins, and asks what are their distinguishing molecular features against the rest of the IDR-ome. This provides them with new information on how nucleolus proteins maintain three phases as a condensate. Zarin et al. (2021) takes the function-first approach further, starting from *conserved* molecular features in IDRs. Then, for any group of IDRs (often from a group of proteins such as a GO term), it trains a classification model that says which of the conserved molecular features have the best predictive power for this group. The particular classification method is named FAIDR (Feature Analysis of IDRs).

Initially, FAIDR was released as a R script (Zarin et al. 2021) targeting the yeast proteome. Later this same script was used in Pritišanac et al. (2026) on the human proteome with 144 molecular features. It remains challenging for an investigator to pull the relevant data (in particular the conserved molecular feature vectors of an entire IDR-ome), run it through the script, and explore interesting and new protein groups. In this project, we ask if we can build a fast and easy-to-use web app to explore the human IDR-ome using the FAIDR statistical model, whether it can reproduce known results, and whether it surfaces useful biological information on new protein groups.

Results

Reproducing the FAIDR statistical model with constrained compute

To check the viability of a web app leveraging FAIDR, we first gauge its compute requirements. One goal for the app is that it is not tied to a particular infrastructure, so that it is easy to host and run into the future. This constrains it to run on users' machines. For an exploration app, the interface should be snappy and take at most a few seconds for computation. We take a group of proteins from Pritišanac et al. (2026) that serves as example, namely proteins found in the nucleolus (GO:0005730), and fit with the same parameters as in the paper (see Materials and methods). The outcome is that we get the same conserved molecular feature strengths in about 6 s (Table 1). This tells us that computation for a single group of proteins is viable, but not of many groups of proteins if we do not want the user to be waiting for minutes.

Table 1: Compute time per pipeline (including final refit) on a M4 MacBook, GO nucleolus example.

Pipeline	Time (s)
Full FAIDR (EM)	5.90
Simplified FAIDR	0.23

We next check if we can reduce that compute budget further. After all, 6 s does not exactly make for a snappy interface, and because this timing is with a highly optimized implementation from specialized statistical software (glmnet in R), our web app would in actuality likely be slower. Hence we also run a degraded pipeline compared to Pritišanac et al. (2026): in addition to the “full” pipeline running 100 individual fits to account for multi-IDR proteins, we also run a “simplified” pipeline made of a single L1-penalized logistic regression (see Materials and methods). For fair comparison, both models are fit so as to produce the same number of molecular features (15). Table 1 shows that the simplified pipeline is roughly 30 times faster, at ~ 0.2 s. It is clear that the single L1-penalized logistic regression requires much less compute than the full FAIDR pipeline.

The simplified FAIDR pipeline gives adequate conserved molecular feature strengths and predictions for function

We hypothesize that the simplified pipeline gives similar molecular features and has similar predictive power than the full FAIDR pipeline from Zarin et al. (2021) (when using the model to predict function for the entire IDR-ome). This is not a given since we are running a simpler algorithm. In particular, no special consideration is given to proteins with multiple IDRs (unlike in the full pipeline), and we assume during fitting that the conserved molecular features of one IDR in the initial group are equally relevant than that of another IDR. To test this, we again use the example GO nucleolus group tuned to get 15 molecular features. The strength (absolute value) of conserved molecular features (+ if enriched, - if depleted) for nucleolus IDRs is shown in Table 2. We see that the two models share twelve of their fifteen features and agree in sign on every shared one, with close values on the strongest: nuclear import (5.69 vs 5.08), the PKB phosphosite (-4.14 vs -4.90), and the lysine and glutamate-repeat terms (3.53 vs 2.74 and 1.43 vs 1.46).

Table 2: Molecular feature statistics, full EM pipeline (t-stat) vs. simplified pipeline (Wald z), GO nucleolus. We verify that the t-stats are correct by comparing to those reported in the supplementary information of Pritišanac et al. (2026) (see Materials and methods).

Feature	Simplified (z)	Full (t)
TRG_NLS_MonoExtN_4	5.69	5.08
SCD	3.82	NA
K_homorep	3.53	2.74
basic	2.85	4.09
E_homorep	1.43	1.46
kappa*	NA	1.31
chain_expanding	1.10	0.37
id=REP_E2 E{2,2}	0.89	0.67
FCR	0.82	1.54
id=REP_K2 K{2,2}	0.15	0.99
AA_K	-0.09	NA
DOC_WW_Pin1_4	-0.89	NA
KL_hydrophathy	-1.19	-0.43
AA_S	-2.48	-2.99
MOD_DYRK1A_RPxSP_1	-2.69	-2.84
AA_I	NA	-2.69
MOD_NMyristoyl	NA	-3.03
MOD_PKB_1	-4.14	-4.90

In addition, both fitted models then make predictions on which IDR should belong to the group of interest (even when not part of the initial group). Crucially, the predictions are at finer IDR granularity, not whole protein level. Both models are calibrated at 1% false positive rate (FPR) to limit the number of false positives for when testing in an actual lab experiment (Pritišanac et al. 2026). The resulting true positive rate (TPR, or recall, how many IDRs predicted positive and recovered from the initial group) gives a sense of the strength of the association captured by the model. Table 3 shows 24.1% recall for the full pipeline, and 21.6% for the simplified one, giving only a slight advantage to the full pipeline. Out of the 48 recovered IDRs in the full pipeline, 42 are found by the simplified pipeline. Taken together, these results suggest that the simplified pipeline doing a single L1-penalized logistic regression performs approximately as well as the full FAIDR pipeline.

(Note that we are not too concerned with bias from overfitting as these models are already heavily penalized to the same number of 15 features versus 144 originally, and we compare at positive sets of same size.)

Table 3: True positive rate at 1% false positive rate, IDR-level, GO nucleolus.

Pipeline	TPR @ 1% FPR
Full FAIDR (EM)	0.241
Simplified FAIDR	0.216

A web app computing conserved molecular feature strengths and predictions for function

Next we endeavor to actually make the simplified FAIDR pipeline usable within a web app. The biggest hurdle is reproducing the L1-penalized logistic regression functionality of the reference implementation (glmnet), and having it run in the browser. The browser has a limited number of ways to execute code. It executes JavaScript, or it executes WebAssembly. We experiment with a fungible JavaScript implementation, that we specify at high level but ask AI to generate in order to capture as many of the subtleties of the algorithm as possible; a few of them we do not intimately know (see Discussion). The result is wired in a web interface, available at <https://moses-lab.github.io/faidr-fit/>; Figure 1 shows a screenshot reproducing the GO nucleolus example, which can be compared to the simplified pipeline in specialized statistical software on three axes. First, the conserved molecular feature strengths: they match those from the simplified pipeline in Table 2. Second, recall, at 21.6%, matches that found in Table 3. A button allows to download predictions for each of the individual IDRs of whether it belongs with the initial group of IDRs specified in input. Third, it takes 1.93 s to run (still on the same machine). It’s a full order of magnitude slower than in Table 1 (1.93 s vs ~0.2 s), but still within our budget of a few seconds. Together, these results show the practicality of having the (simplified) FAIDR pipeline run in the browser.

CPU cache locality matters in JavaScript execution

In one round of AI implementing our specific L1-penalized logistic regression (see Discussion), we find code as in Listing 1. The nested loop reads as a matrix-vector multiplication, as would happen during updating of the linear predictor η in the L1-penalized logistic regression algorithm (Hastie, Tibshirani, and Wainwright 2015). The inner loop computes a row-column

FAIDR-Fit

Explore the molecular features and function of intrinsically disordered regions (IDRs) across the human proteome in your browser

Define function by specifying any group of related proteins or IDRs

19,032 IDRs from 11,640 proteins available

Q99575_IDR_721_799
P55265_IDR_576_607
075691_IDR_1695_1763
Q8NHQ9_IDR_570_600
Q9GZR7_IDR_261_383
Q9ULW3_IDR_1_41
075691_IDR_2572_2613

Or choose a predefined group

Nucleolus (199 IDRs) ▾

Or randomize a number of IDRs

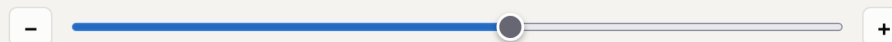
199

Randomize

GO:0005730 nucleolus · 122 proteins / 199 IDRs · computed in 1.93 s

RESULTS

Number of molecular features



15 molecular features selected

Recall: 21.6% Precision (Enrichment): 18.6% (17.8×)

19,032 IDR function predictions available for download

Download predictions (.tsv)

■ enriched (+z) ■ depleted (-z)

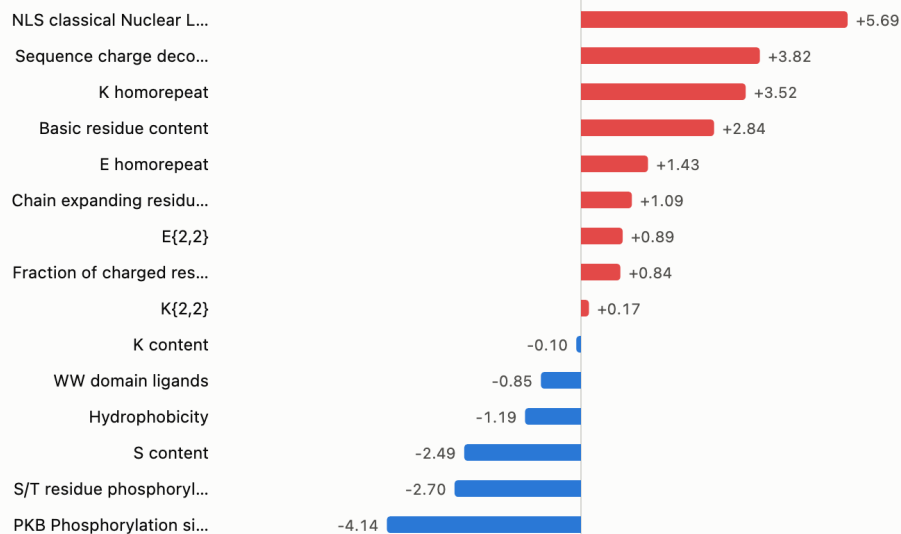


Figure 1: Screenshot of the app matching the simplified pipeline values from the specialized statistical software. In particular: the conserved molecular feature strengths (Table 2), and recall at 21.6% (Table 3).

dot product to get the corresponding coefficient for η . It uses column-major storage (column as first dimension for the matrix).

Listing 1 AI-generated matrix-vector multiplication implementation, initial attempt

```
const eta = new Float64Array(n);
for (let i = 0; i < n; i++) {
  let e = beta0;
  for (let j = 0; j < p; j++) e += X[j][i] * beta[j];
  eta[i] = e;
}
```

We hypothesize this code to be inefficient. Indeed, the whole point of column-major storage is to be able to compute η incrementally, as the sum of column vectors $\mathbf{x}_j\beta_j$. We test its efficiency by swapping inner and outer loops, making the inner one varying only in i (Listing 2), and sample execution times for both versions (see Materials and methods for more details). Table 4 shows timings: at 20,000 observations, column-order with 1,000 features gets a 5x speedup over row-order, and a 2x speedup with 144 features (our use case). In conclusion, our contained JavaScript implementation benefits from the usual “tricks of the trade” in numerical algorithm implementation. It is surprising that AI would do the inefficient thing; we cover our approach to countering the behavior in Discussion.

Listing 2 Matrix-vector multiplication rewritten. Eight 64-bit column elements are now accessed from a 64-byte L1 cache line in the inner loop, and betaj is pinned to a register.

```
const eta = new Float64Array(n);
eta.fill(beta0);
for (let j = 0; j < p; j++) {
  const col = X[j];
  const betaj = beta[j];
  for (let i = 0; i < n; i++) {
    eta[i] += col[i] * betaj;
  }
}
```

Table 4: Matrix-vector multiplication timing by memory access pattern (95% CIs over 25 samples)

n	p	Row-order (ms)	Column-order (ms)	Speedup
20000	1000	66.535 ± 0.712	12.473 ± 0.036	5.33x
20000	144	3.643 ± 0.086	1.722 ± 0.018	2.11x

The app reports valid biology on new datasets

While the app is validated against known protein groups already studied with FAIDR, its predictive power should extend to groups of proteins established by unrelated means. We obtain such a dataset from mass spectrometry experiments. One group of proteins is labeled “splicing”, another one is labeled “cajal_body”. The splicing group contains 167 IDRs and an enrichment factor of 18.2x, with the top conserved molecular feature being K{2,2}, two consecutive lysines (Figure 2). The cajal body group contains 151 IDRs but an enrichment factor of 2.6x. To get a sense of the model’s typical enrichment at this size group and number of features, we sample 150 random IDRs (See Randomize button in Figure 2) a few times and find enrichment varying between 2x-6x. For the splicing group, we download the predictions made by the model and find 189 additional IDRs predicted to belong to that group (based on conserved molecular feature strengths), the top three of which are Q13427_IDR_179_754, Q5T200_IDR_1_1515, and Q9Y2W1_IDR_1_955. Uniprot reports all three proteins (Q13427, Q5T200, Q9Y2W1) to be involved in mRNA splicing. In the amino acid sequence range of the IDRs (179-754, 1-1515, 1-955), we find 11, 14, and 6 lysine-lysine pairs.

The results suggest that for this particular cajal body group, if the app works, the IDRs do not share evolutionary conserved molecular features. For the splicing group, the app correctly flags additional proteins involved in splicing, as validated by Uniprot. Further, the top molecular feature checks out with the amino acid sequences of those proteins’ IDRs: if each amino acid were equally likely we would have expected only $1/20 \cdot 1/20 \cdot 999 \approx 2.5$ lysine-lysine pairs on average over 1000 amino acids; yet we observe higher rates.

Discussion

We have demonstrated that a statistical model of biological data, previously restricted to specialized software (R), can be brought into an interactive web app. One particular feature of the UI is varying the number of molecular features in the model on the fly, quickly allowing

FAIDR-Fit

Explore the molecular features and function of intrinsically disordered regions (IDRs) across the human proteome in your browser

Define function by specifying any group of related proteins or IDRs

19,032 IDRs from 11,640 proteins available

Q01081
Q53GS9
Q86VM9
Q9NP64
Q96NC0
Q969S3
Q95218

Or choose a predefined group

— choose a group —

Or randomize a number of IDRs

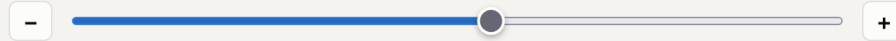
167

Randomize

167 IDRs matched from 88 lines · computed in 1.37 s

RESULTS

Number of molecular features



14 molecular features selected

Recall: 21.6% Precision (Enrichment): 16.0% (18.2x)

19,032 IDR function predictions available for download

Download predictions (.tsv)

■ enriched (+z) ■ depleted (-z)

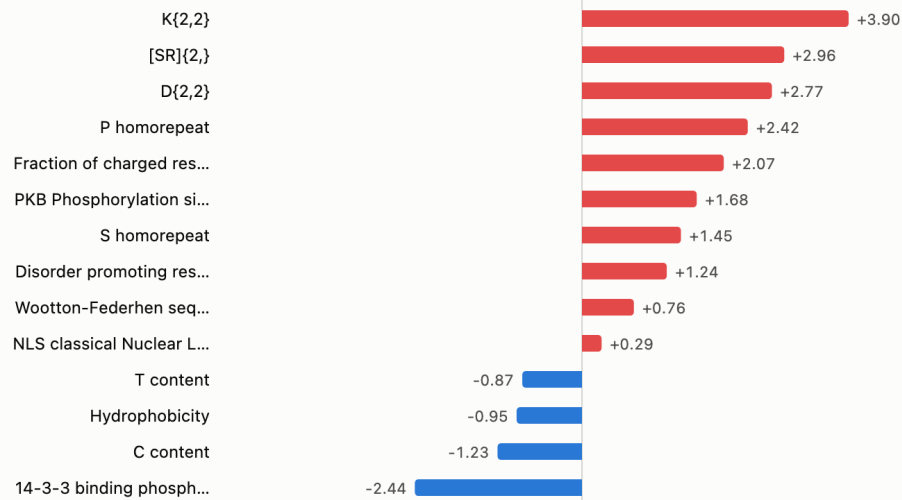


Figure 2: App output for the splicing group: enrichment score and K{2,2} as the top conserved feature.

to get a sense of which ones matter most for a given function. Distinctively, the web page is entirely self-contained and requires no infrastructure; in fact, the page can be saved locally and the app continues working. The statistical model runs directly in the browser.

Partly, this is possible by leveraging AI for coding assistance (Anthropic’s Opus 4.8 and Sonnet 5). Building an app like this requires bringing several techniques together. Evidently, web development coding skills are required; we’ve found AI to be great at it working off high level descriptions. In terms of data, the 19,032 x 144 molecular features need to be embedded in the page. Leveraging AI for rapid coding, we are able to: 1) quantize the molecular features to 8 bits (which turns out to be sufficient precision), 2) compress the resulting stream, 3) base64-encode the compressed stream for embedding in the page, 4) have all the inverse steps so that the app can make use of the data. The end result is a page only 3 Mbytes large, quick to download on modern communication networks including mobile. While AI has certainly sped up development for us, it is not without pitfalls, a topic we discuss next.

Bringing the statistical algorithm to the browser with AI is delicate. We initially task Opus 4.8 to write a L1-penalized logistic regression JavaScript library in the style of Friedman, Hastie, and Tibshirani (2010) and Hastie, Tibshirani, and Wainwright (2015) and the glmnet reference implementation in R. We find three main issues on this implementation (and/or the next few ones), that we look at in turn. First, the implementation sometimes fails to converge. This is due to using an absolute convergence threshold check instead of a relative one. Second, some of its tests seem to simply reimplement the functionality, leading to low confidence in testing (what if it is incorrect the same way twice, in the library code and in the test?). Third, we suspect performance to be lacking beyond the penalty of running in a JavaScript engine; one issue is its incorrect use of column-major storage as demonstrated in Results, others are related to missing subtle algorithmic optimizations from the reference implementation (step-halving, coordinate descent only on the active set of coefficients).

As a result, while enthusiastic of AI’s usefulness, we grow more cautious of its claims, outputs, and reflect on our usage of it. For the JavaScript library specifically, we adopt two strategies. The first one is to not incrementally fix AI’s output. Instead we write a specification of what the inputs and outputs are, the constraints, the pitfalls to avoid (what not to do), and ask AI to reimplement the library entirely every time from a refined prompt. Not only does it prevent code accretion over time, it more so helps clarify our own thinking of what is relevant to specify. The prompt for the final implementation is given in the Appendix. The second strategy is to validate the implementation against a reference oracle, in other words, relying on testing outside of that produced by AI. We test sample cases against R’s reference implementation (see Materials and methods).

In aggregate, we find working with AI makes it very easy to become too hands-off and

too far removed from the details. Sometimes the details matter. A good compromise as a user is at a minimum glueing together the pieces the AI produces (this is what indirectly led to finding out the inefficient column-major computation). Our experience is that we are feeling more in control and end up with better understanding using Anthropic’s free tier (with non-agentic limited-use Sonnet 5), than with the pro tier we started with (Opus 4.8 without apparent limit). It will be interesting to see if open-weight models on consumer hardware become powerful enough to be compelling versus today’s proprietary frontier.

Future work can reuse the JavaScript library prompt and ask AI to generate a WebAssembly implementation instead of a JavaScript one and find out if WebAssembly is more performant. (Since inputs and outputs are part of the specification prompt, the implementations should in principle be swappable.) Another interesting line of inquiry is whether the web app is easily updatable (with or without AI), assuming demand from more users exposed to the FAIDR statistical model. A tentative upgrade is having the proteomes for more species beyond just humans.

Materials and methods

Obtaining molecular feature conservation vectors for IDRs in the human proteome

We use the data from Pritišanac et al. (2026). It is made available from a ZENODO repository, with a sidecar python script for downloading and unpacking available at https://github.com/IPritisanac/IDR_ES/blob/main/download_zenodo.py. The following command produces the extracted ZENODO folder locally:

```
python3 download_zenodo.py --target ./ZENODO
```

The GO nucleolus example

With the ZENODO folder available locally, we run the following R script, which reproduces the FAIDR EM pipeline from Zarin et al. (2021), and adds a simplified FAIDR pipeline.

Pipeline comparison (complete R script)

```

#!/usr/bin/env Rscript
# Full FAIDR EM pipeline vs. simplified single-lasso pipeline,
# GO nucleolus, fit on the whole IDR-ome. One run of each, timed
# end-to-end (incl. final refit).
#
# Full    = paper's algorithm: fixed lam = 0.2, 20 EM iterations
#          x 5 IRLS steps, feature t-stats from an unpenalised
#          WLS refit of the working response z.
# Simple = one L1-penalized logistic fit, path stopped at
#          dfmax = 30, Wald z-stats from an unpenalised logistic
#          refit on the model with 15 features.

suppressPackageStartupMessages(library(glmnet))

ES_FILE <- "ZENODO/MAP/HUMAN_ES.txt"
GO_FILE <- "ZENODO/HIGH_AUC_PPV_FILTERED_FUNCTIONS/G00005730_nucleolus.txt"

load_X <- function() {
  es <- read.delim(ES_FILE, check.names = FALSE)
  es <- es[es$GID != "EWEIGHT", ]
  feat <- setdiff(colnames(es), c("GID", "IDRID", "NAME", "GWEIGHT"))
  X <- as.matrix(sapply(es[, feat], as.numeric))
  rownames(X) <- es$IDRID
  X[is.na(X)] <- 0
  X
}

align_y <- function(X) {
  tg <- read.delim(GO_FILE, check.names = FALSE)
  as.integer(tg[["G00005730_nucleolus"]][match(rownames(X), tg$idr_name)])
}

X      <- load_X()
y      <- align_y(X)
idr_names <- rownames(X)

```

```

gene_groups <- function(names)
  split(seq_along(names), sapply(strsplit(names, "_"), `[, 1])

# --- full FAIDR pipeline -----
faidr_full <- function(X, y, idr_names, lam = 0.2, niter = 20, msteps = 5) {
  g2r <- gene_groups(idr_names)
  multi <- which(lengths(g2r) > 1)
  w <- numeric(length(y))
  for (g in seq_along(g2r))
    w[g2r[[g]]] <- 1 / length(g2r[[g]]) # 1 if single IDR

  pr <- NULL; w1 <- NULL; z <- NULL
  mod <- glmnet(X, y, family = "gaussian", weights = w)
  for (i in 1:niter) {
    for (j in 1:msteps) { # M-step: IRLS as weighted gaussian lasso
      eta <- predict(mod, X, s = lam)
      pr <- 1 / (1 + exp(-eta))
      w1 <- pr * (1 - pr); w1[w1 == 0] <- exp(-20)
      z <- eta + (y - pr) / w1
      mod <- glmnet(X, z, family = "gaussian", weights = w1 * w)
    }
    for (g in multi) { # E-step: responsibility weights
      rows <- g2r[[g]]
      w[rows] <- y[rows] * pr[rows] + (1 - y[rows]) * (1 - pr[rows])
      w[rows] <- w[rows] / sum(w[rows])
    }
  }
  for (g in multi) { # final y-independent weights
    rows <- g2r[[g]]
    w[rows] <- (pr[rows] + 0.05) / sum(pr[rows] + 0.05)
  }
  b <- as.numeric(coef(mod, s = lam))[-1]
  names(b) <- colnames(X)
  sel <- names(b)[b != 0]
  refit <- glm(z ~ X[, sel], weights = w1 * w)

```

```

tstat <- setNames(rep(NA_real_, ncol(X)), colnames(X))
tstat[sel] <- summary(refit)$coefficients[-1, "t value"]
list(coef = b, stat = tstat, nsel = length(sel),
      score = as.numeric(predict(mod, X, s = lam)))
}

# --- simplified pipeline: single lasso + unpenalized refit -----
faidr_simple <- function(X, y, target = 15) {
  fit <- glmnet(X, y, family = "binomial", alpha=1, nlambda=300, dfmax=30)
  cmat <- as.matrix(coef(fit))
  idx <- which.min(abs(fit$df - target))
  b <- cmat[-1, idx]
  sel <- names(b)[b != 0]
  refit <- glm(y ~ X[, sel], family = binomial())
  zstat <- setNames(rep(NA_real_, ncol(X)), colnames(X))
  zstat[sel] <- summary(refit)$coefficients[-1, "z value"]
  list(coef = b, stat = zstat, nsel = length(sel),
        score = as.numeric(cmat[1, idx] + X %*% b))
}

# --- run + time both (timings include the final refit) -----
t_full <- system.time(res_full <- faidr_full(X, y, idr_names))["elapsed"]
t_simple <- system.time(res_simple <- faidr_simple(X, y))["elapsed"]
speedup <- round(t_full / t_simple)

if (sys.nframe() == 0) {
  cat(sprintf("full: %d features, %.2fs\n", res_full$nsel, t_full))
  cat(sprintf("simple: %d features, %.2fs\n", res_simple$nsel, t_simple))
  cat(sprintf("speedup: %dx\n", speedup))
}

```

T-stats as found in the original paper's supplementary information (complete R script)

```
paper_tstats <- function(cdt_file =
  "ZENODO/Tstats_top_148_from_full_training.cdt") {
  ref <- read.delim(cdt_file, check.names = FALSE)
  ref <- ref[!(ref$GID %in% c("AID", "EWEIGHT")), ]
  pcol <- as.numeric(ref[[grep("G00005730_nucleolus", colnames(ref))]])
  x <- setNames(pcol, gsub("[^A-Za-z0-9]+", ".", ref$NAME))
  x <- x[x != 0]
  sort(x, decreasing = TRUE)
}

x <- paper_tstats()
cat(sprintf("%-20s %9.4f\n", names(x), x), sep = "")
```

True positive rate (recall) evaluation (R snippet)

```
recovered <- function(score, y, fpr_target = 0.01) {
  thr <- quantile(score[y == 0], probs = 1 - fpr_target, na.rm = TRUE)
  idr_names[y == 1 & score > thr]
}

tpr_at_fpr <- function(score, y, fpr_target = 0.01)
  length(recovered(score, y, fpr_target)) / sum(y == 1)
```

Testing the impact of CPU cache locality in matrix-vector multiplication memory access patterns

The core library code and app code are available at <https://github.com/moses-lab/faidr-fit/>. There are no dependencies.

The two matrix-vector multiplication implementations are wrapped into a script for easy reproducibility.

For the default 20,000 observations and 1,000 features, run:

```
cd core/  
node scripts/matvec_benchmark.js
```

For 144 features:

```
node scripts/matvec_benchmark.js 20000 144
```

The script does 5 warm-up runs followed by 25 runs whose times are sampled, for each implementation in turn.

Testing the core library against R's reference (glmnet)

From the same code repository:

```
cd core/  
node --test test/lambda-max.test.js  
node --test test/predict.test.js
```

These in fact check the core library computations against values in json files at `test/fixtures`. The fixture files are generated out of the test files above with:

```
node scripts/generate-oracles.js
```

Briefly, it scans for R-ORACLE-TAG-START/R-ORACLE-TAG-END tags in the JavaScript test files, runs the sample case delimited by them in R via Rscript, and captures the values output by R in the fixture files. If you modify one such test case, you need to run the generate-oracles script again so that the fixture files reflect the updated values.

Building the app from data

From the same code repository:

```
cd app/  
node prep-data.js path/to/ZENODO/MAP/HUMAN_ES.txt 8  
node prep-go.js path/to/ZENODO/HIGH_AUC_PPV_FILTERED_FUNCTIONS/  
node bundle.js
```

The first script mainly builds the IDR-ome molecular feature vectors into a dense representation quantized to 8 bits. The second one prepares the predefined set of GO terms (groups of proteins). The outputs of both the first and second scripts are written under a .cache folder, which is included in the code repository. Because of it, the third script can be run without the first two; it packs the code and data into the final 3 Mbytes self-contained page.

References

- Conicella, Alexander E., Gül H. Zerze, Jeetain Mittal, and Nicolas L. Fawzi. 2016. “ALS Mutations Disrupt Phase Separation Mediated by α -Helical Structure in the TDP-43 Low-Complexity C-Terminal Domain.” *Structure* 24 (9): 1537–49. <https://doi.org/10.1016/j.str.2016.07.007>.
- Friedman, Jerome H., Trevor Hastie, and Rob Tibshirani. 2010. “Regularization Paths for Generalized Linear Models via Coordinate Descent.” *Journal of Statistical Software* 33 (February): 1–22. <https://doi.org/10.18637/jss.v033.i01>.
- Hastie, Trevor, Robert Tibshirani, and Martin Wainwright. 2015. *Statistical Learning with Sparsity: The Lasso and Generalizations*. New York: Chapman and Hall/CRC. <https://doi.org/10.1201/b18401>.
- King, Matthew R., Kiersten M. Ruff, Andrew Z. Lin, Avnika Pant, Mina Farag, Jared M. Lalmansingh, Tingting Wu, et al. 2024. “Macromolecular condensation organizes nucleolar sub-phases to set up a pH gradient.” *Cell* 187 (8): 1889–1906.e24. <https://doi.org/10.1016/j.cell.2024.02.029>.
- Pritišanac, Iva, T. Reid Alderson, Đesika Kolarić, Taraneh Zarin, Shuting Xie, Alex Lu, Aqsa Alam, et al. 2026. “A Functional Map of the Human Intrinsically Disordered Proteome.” *Proceedings of the National Academy of Sciences* 123 (21): e2604562123. <https://doi.org/10.1073/pnas.2604562123>.
- Ruff, Kiersten M., Matthew R. King, Alexander W. Ying, Vicky Liu, Avnika Pant, Whitney E. Lieberman, Min Kyung Shinn, Xiaolei Su, Cigall Kadoch, and Rohit V. Pappu. 2026. “Molecular Grammars of Predicted Intrinsically Disordered Regions That Span the Human Proteome.” *Cell* 189 (1): 323–342.e17. <https://doi.org/10.1016/j.cell.2025.10.019>.
- Zarin, Taraneh, Bob Strome, Alex N Nguyen Ba, Simon Alberti, Julie D Forman-Kay, and Alan M Moses. 2019. “Proteome-Wide Signatures of Function in Highly Diverged Intrinsically Disordered Regions.” Edited by Michael B Eisen, Xavier Darzacq, and Jessica Siltberg-Liberles. *eLife* 8 (July): e46883. <https://doi.org/10.7554/eLife.46883>.
- Zarin, Taraneh, Bob Strome, Gang Peng, Iva Pritišanac, Julie D Forman-Kay, and Alan

M Moses. 2021. "Identifying Molecular Features That Are Associated with Biological Function of Intrinsically Disordered Protein Regions." Edited by Michael B Eisen. *eLife* 10 (February): e60220. <https://doi.org/10.7554/eLife.60220>.

Appendix

JavaScript library prompt

Our goal here is to build a pure, self contained, JavaScript, penalized (lasso) logistic regression library. The constraints are as follows.

General

- It should follow conventions in Friedman et al's 2010 "Regularization Paths for Generalized Linear Models via Coordinate Descent" and Hastie et al's 2015 "Statistical Learning with Sparsity"
- As corollary, it should follow the conventions adopted in the glmnet's implementation
- It should focus on L1 penalized logistic regression only
- It should be as fast as possible
- It should be very readable, especially to a human reviewer
 - e. g., avoid one liner standard deviation loops in the middle of a function
- It should not have arbitrary thresholds in the code

Public API

```
## `fitLassoLogistic(X,y,opts={})`
```

Given a design matrix `X` (given column-major), and 0/1 labels in vector array `y`, compute a L1 penalized logistic regression fit. Not just one, but an entire lambda path, starting at $\lambda_{\max}$. It is essential to exploit warm-start for maximum performance.

`opts`:

- `nlambda`: the number of desired lambdas in a (full) lambda path
- `dfmax`: stop path once we have a fit with this many features selected

Returns:

- The lambda path. First element should be lambda max.
- A corresponding df array of number of selected features.
- A corresponding array with fitted coefficients.

```
## `fitLogistic(X,y,opts={})`
```

`X` is again a design matrix, but it's expected to be one with only desired features (first selected with `fitLassoLogistic`). `fitLogistic` is a plain logistic regression (not penalized). Its goal is to get proper z statistics for the strength of features and should return those.

Returns:

- beta0 and beta array, the coefficients
- waldZ array of z statistics

```
## `predictLogistic(fit,newx,lambda_idx)`
```

Returns:

- The log odds (eta vector) predicted by the coefficients at lambda_idx within the fit object.
- The actual lambda value at that idx.

The actual lambda value will be more for inspection/debugging.

Behaviour

The numerical stability is super important. Glmnet has additional techniques to avoid divergence, these need to be in as well. Pieces that were identified:

- Converged threshold needs to be relative
- Step-halving for both fast convergence and preventing overshooting
- Active set optimization so we're only doing coordinate descent on non zero coefficients

Testing

Write a few tests checking major invariants of the algorithm. Use node's

native test() framework.

Gotchas

All matrices are expected to be given (and used internally) in column-major order. In addition, all matrix-matrix, matrix-vector multiplications should exploit that ordering for maximum cache locality. This is an example of what **NOT** to do:

```
const eta = new Float64Array(n);
for (let i = 0; i < n; i++) {
    let e = beta0;
    for (let j = 0; j < p; j++) e += Xstd[j][i] * beta[j];
    eta[i] = e;
}
```